

AI 2002

Artificial Intelligence

Lecture 3

Mahzaib Younas

Lecture Department of Computer Science

FAST NUCES CFD

State Representation

State

A state is the representation of the physical configuration. It may be considered as all relevant aspects of the problem.

- Three ways to represent states and the transitions between them.
- Atomic Representation
- Factored Representation
- Structured Representation

Atomic Representation

A state (such as B or C) is a black box with no internal structure

The algorithm underlying search, game-playing, Markov Models, and Markov decision process etc.

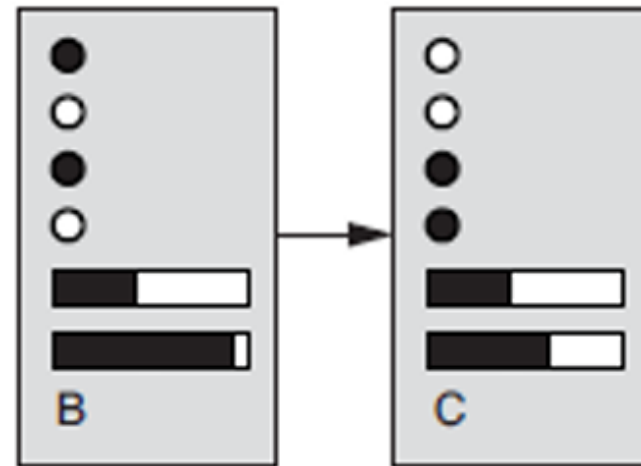


Factored Representations

A state consists of a vector of attribute values, values can be Boolean, real-valued, or on of a fixed set of symbols.

Example:

- Constraint Satisfaction problem
- Propositional Logic
- Planning

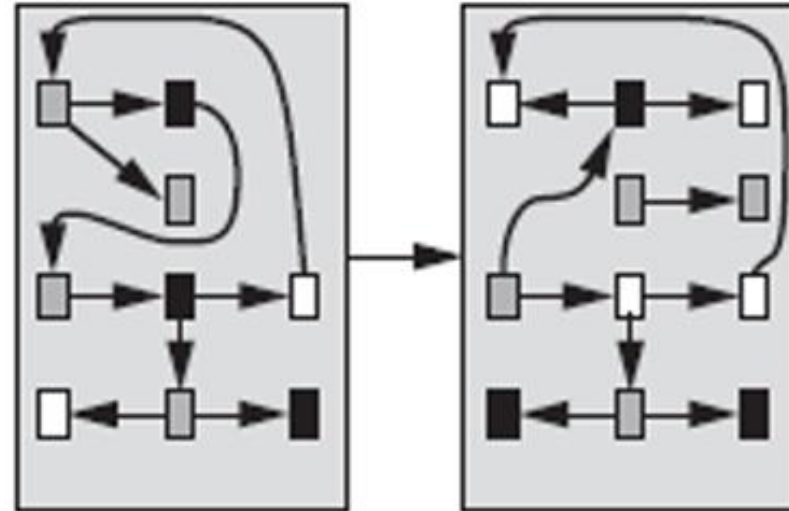


Structured Representation

A state includes objects, each of which may have attributes of its own as well as relationships to other objects.

Example:

- First Order Logic
- Knowledge based Learning



Searching

- How can we find a sequence of actions that achieves its goal when no single action will do?
- One general approach to problem solving is
 - To reduce the problem to be solved to one of a searching a tree of graph.
- Search plays a key role in many part of AI.

Problem Solving Agents

- A type of goal-based agent decide what to do by finding sequences of actions that lead to desirable states
- Problem solving agents use atomic representation.
- It based on two things
 - Goal Formulation
 - Problem Formulation

Goal-based agents that use more advanced factored or structured representation are usually called planning agents.

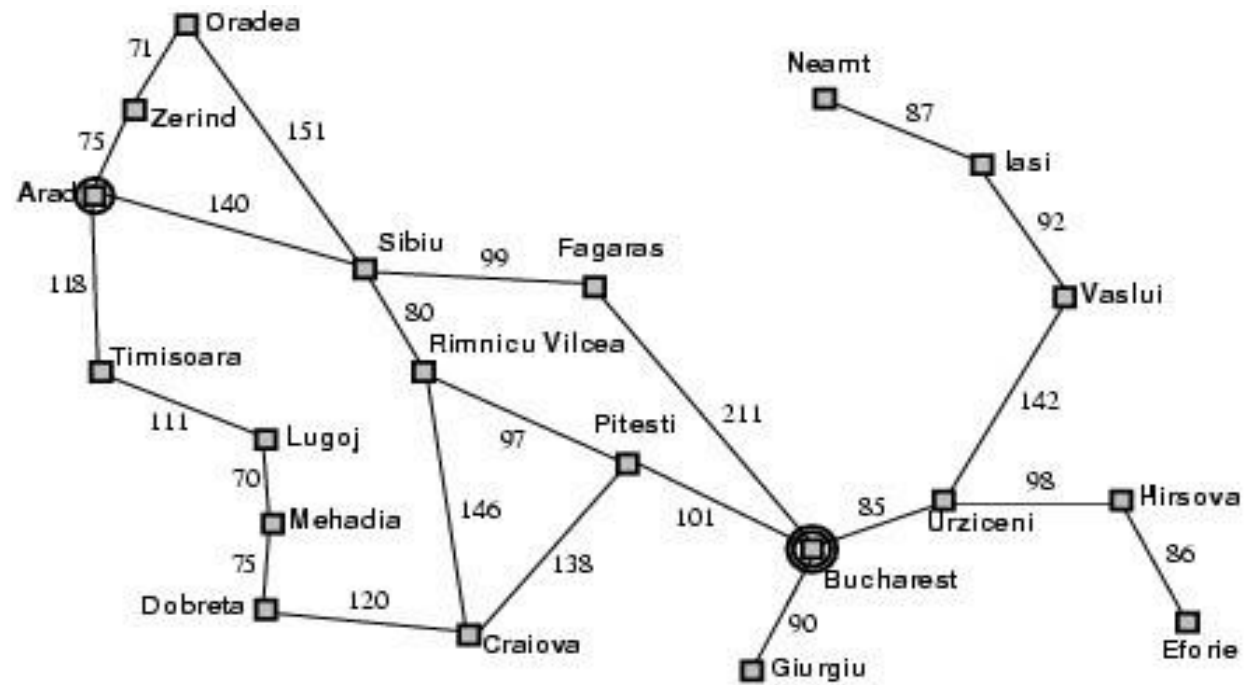
Problem Solving Agents (Cont...)

- Goal Formulation
 - based on current situation and agent's performance measure
- Problem Formulation
 - deciding what actions and states to consider, given a goal

The process of looking for such a sequence of actions is called search

Example: Romania Touring

On holiday the agent is in Romania visiting in Arab. Flight Leaves tomorrow from Bucharest.



Solution

1. Formulate goal (perf. evaluation):
be in Bucharest before the flight
2. Formulate problem:
states: various cities
actions: drive between cities
3. Search:
sequence of cities

Problem Solving Agent

```
function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action
  persistent: seq, an action sequence, initially empty
               state, some description of the current world state
               goal, a goal, initially null
               problem, a problem formulation

  state ← UPDATE-STATE(state, percept)
  if seq is empty then
    goal ← FORMULATE-GOAL(state)
    problem ← FORMULATE-PROBLEM(state, goal)
    seq ← SEARCH(problem)
    if seq = failure then return a null action
  action ← FIRST(seq)
  seq ← REST(seq)
  return action
```

Well Defined Problem:

- A problem can be defined formally by five components:
 - Initial state
 - Actions
 - Transition model: description of what each action does (successor)
 - Goal test
 - Path cost

Problem Solving Agent (Cont...)

Actions

- A description of possible actions available to agent.
- Given particular state s , Action(s)
- Action(s), is applicable for s .
- Example:
 - From state In(Arad) our actions can be
 1. Go (Sibiu)
 2. Go (Timisoara)
 3. Go (Zerind)

Problem Solving Agent (Cont...)

Transition Model: A description of what each step does; is called the transition model.

Example:

$\text{Result}(\text{In}(\text{Arad}), \text{Go}(\text{Zerind})) = \text{In}(\text{Zerind})$

State

Action

Successor

Problem Solving Agent (Cont...)

Goal Test

- It determine whether a given state is goal state.
- Two types of goal test
 - explicit, e.g. In(Bucharest)
 - set of possible goal states, and the test simply checks whether the given state is one of them. The agent's goal in Romania is the singleton set {In(Bucharest)}.
 - implicit, e.g. checkmate
 - in chess, the goal is to reach a state called "checkmate," where the opponent's king is under attack and can't escape.

• Path Cost

- that assigns a numeric cost to each path
- Example:
 - distance traveled

• Step Cost:

- The step cost of taking action a in state s to reach state s' is denoted by $c(s, a, s')$. The step costs for Romania are shown in Figure as route distances.

Problem Solving Agent (Cont...)

State Space:

- Combine the initial state, action and transition is called the state space of the problem.
- The set of all states reachable from the initial state by any sequence of action

Graph:

- Directed network in which the nodes are state and links between nodes and action.
- Path
 - Path in the state space is the sequence of states connected by sequence of action.

Problem Solving Agent (Cont...)

Solution:

- A solution to a problem is an action sequence that leads from the initial state to a goal state.

Optimal Solution:

- Solution quality is measured by OPTIMAL SOLUTION the path cost function, and an optimal solution has the lowest path cost among all solutions

Problem Abstraction

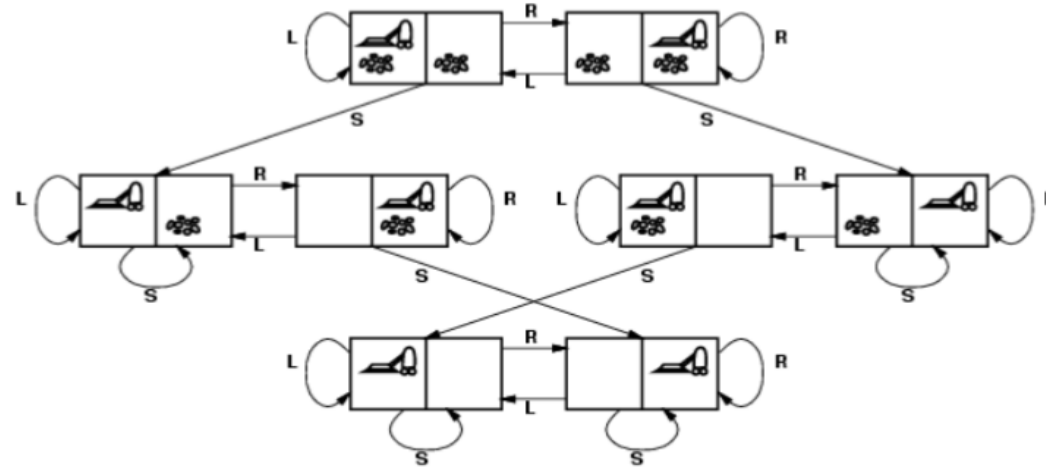
- Real world is complex and has more details

Irrelevant details should be removed from state space and actions, which is called abstraction

- What's the appropriate level of abstraction?
 - Valid
 - the abstraction is valid, if we can expand it into a solution in the more detailed world
 - Useful
 - the abstraction is useful, if carrying out the actions in the solution is easier than the original problem
- remove as much detail as possible while retaining validity and usefulness***

Example: Vacuum Problem

The state space for the vacuum world



states? integer dirt and robot location

actions? *Left, Right, Suck*

goal test? no dirt at all locations

path cost? 1 per action

Explanation

- **States:**

- The state is determined by both the agent location and the dirt locations. The agent is in one of two locations, each of which might or might not contain dirt. Thus there are $2 \times 2^2 = 8$ possible world states. A larger environment with n locations has $n \cdot 2^n$ states.

- **Initial state:**

- Any state can be designated as the initial state.

- **Actions:**

- In this simple environment, each state has just three actions: Left, Right, and Suck. Larger environments might also include Up and Down.

Explanation

- **Transition model:**

- The actions have their expected effects, except that moving Left in the leftmost square, moving Right in the rightmost square, and Sucking in a clean square have no effect. The complete state space is shown in Figure

- **Goal test:**

- This checks whether all the squares are clean.

- **Path cost:**

- Each step costs 1, so the path cost is the number of steps in the path.

Example 2: 8-Puzzle Problem

1. States
2. Initial state
3. Actions
4. Transition model
5. Goal test
6. Path cost.

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

Solution:

The standard formulation is as follows:

- **States**

- A state description specifies the location of each of the eight tiles and the blank in one of the nine squares.

- **Initial state**

- Any state can be designated as the initial state. Note that any given goal can be reached from exactly half of the possible initial states

- **Actions**

- The simplest formulation defines the actions as movements of the blank space Left, Right, Up, or Down. Different subsets of these are possible depending on where the blank is.

Solution:

The standard formulation is as follows:

- **Transition model**

- Given a state and action, this returns the resulting state; for example, if we apply Left to the start state in Figure the resulting state has the 5 and the blank switched.

- **Goal test**

- This checks whether the state matches the goal

- **Path cost**

- Each step costs 1, so the path cost is the number of steps in the path.

Example: Robotic Assembly

- **States**

- real-valued coordinates of robot joint angles; parts of the object to be assembled

- **Actions**

- continuous motions of robot joints

- **Transition model**

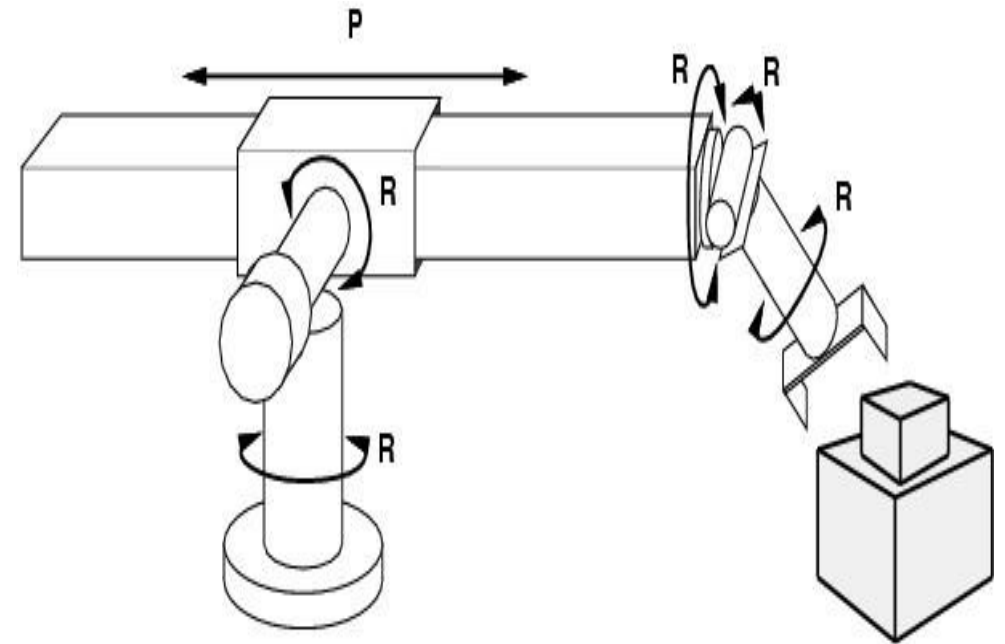
- States of robot joints after each action

- **Goal test**

- complete assembly

- **Path cost:**

- time to execute



Real World Problem

Airline travel problems that must be solved by a travel planning Web site:

Solution:

States:

Each state obviously includes a location (e.g., an airport) and the current time. Furthermore, because the cost of an action (a flight segment) may depend on previous segments, their fare bases, and whether they were domestic or international, the state must record extra information about these “historical” aspects.

Initial state:

This is specified by the user’s query.

Actions

Take any flight from the current location, in any seat class, leaving after the current time, leaving enough time for within-airport transfer if there is a preceding flight segment.

Solution:

Transition model:

The state resulting from taking a flight will have the flight's destination as the current location and the flight's arrival time as the current time.

Goal test:

Are we at the final destination specified by the user.

Path cost:

This depends on monetary cost, waiting time, flight time, customs and immigration procedures, seat quality, time of day, type of airplane, frequent-flyer mileage awards, and so on.

Practice Problem:

1. Touring Problem
2. Travelling salesperson problem
3. VLSI Layout
4. Robot Navigation
5. Automatic Assembly Sequencing

Searching for Solution



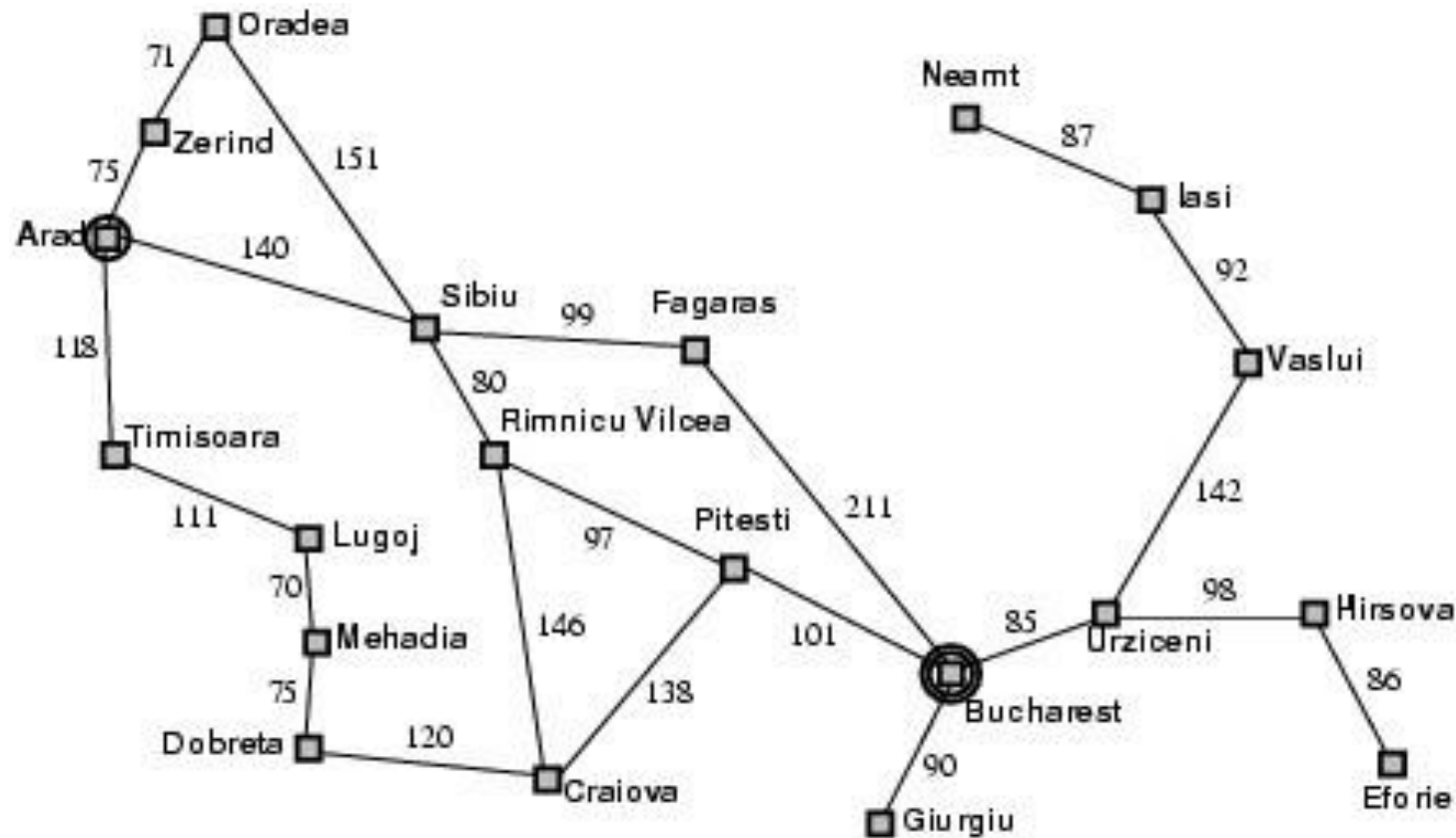
Search Tree

- **Search tree:** generated by initial state and possible actions

Basic idea:

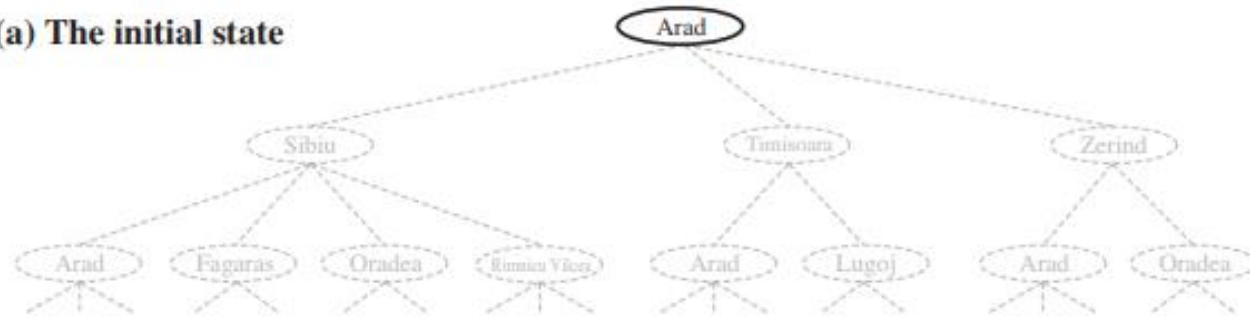
- ❑ offline, simulated exploration of state space by generating successors of already-explored states (expanding states)
- ❑ the choice of which state to expand is determined by search strategy

Tree Search Example: finding a route from Arad to Bucharest.

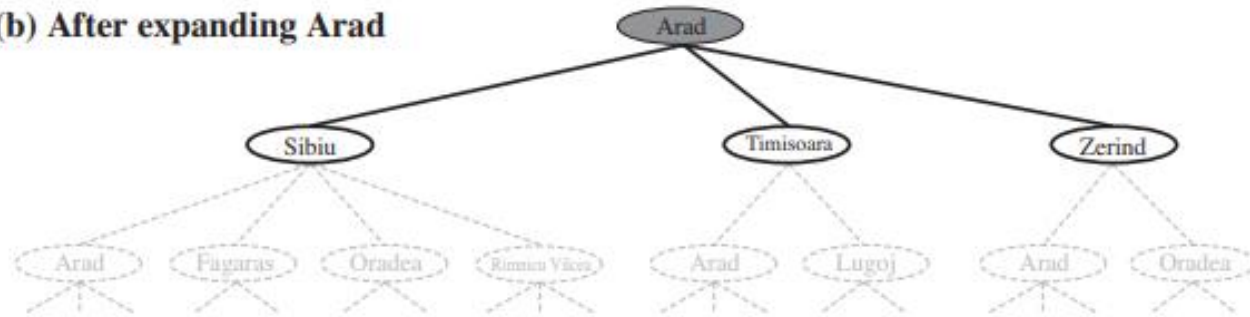


Tree Example (Cont...)

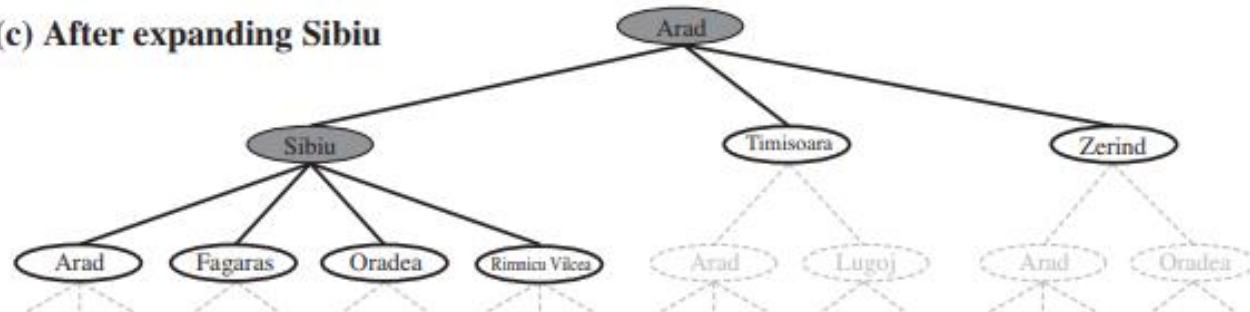
(a) The initial state



(b) After expanding Arad



(c) After expanding Sibiu



Basic Terminologies of Tree Search

- Frontier
 - Set of all lead node available for expansions at any given point
- Repeated State
- Loopy Path
 - Identify the loop path for example arad to sibiu to arad
- Redundant Path
 - More than one way to get from one state to another

General search Tree Algorithm

function TREE-SEARCH(*problem*) **returns** a solution, or failure

 initialize the frontier using the initial state of *problem*

loop do

if the frontier is empty **then return** failure

 choose a leaf node and remove it from the frontier

if the node contains a goal state **then return** the corresponding solution

 expand the chosen node, adding the resulting nodes to the frontier

Explanation:

- Step 1: initialize the frontier using the initial state of the problem
- Step 2: Enter a loop
- Step 3: check if the chosen node contains a goal state
- Step 4: Expand the chosen node

Explanation:

- **Step 2: Enter a loop**

check if the frontier is empty

if the frontier is empty, return failure as no solution is found.

choose a leaf node from the frontier and remove it.

select a node from the frontier based on the chosen search strategy

- **Step 3: check if the chosen node contains a goal state**

- **Step 4: Expand the chosen node**

Explanation:

- **Step 3: check if the chosen node contains a goal state**

if the chose state contains a goal state, return the corresponding solution.

- **Step 4: Expand the chosen node**

Generate new nodes by applying valid actions to the chosen node's state. Add the resulting node to the frontier.

Graph Search Algorithm

```
function GRAPH-SEARCH(problem) returns a solution, or failure
  initialize the frontier using the initial state of problem
  initialize the explored set to be empty
  loop do
    if the frontier is empty then return failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state then return the corresponding solution
    add the node to the explored set
    expand the chosen node, adding the resulting nodes to the frontier
    only if not in the frontier or explored set
```

Basics

We Augment Tree-Search with a explored set, which remembers every expanded node.

